7 - push_swap

minitalk

- **useful links**
  1. **https://github.com/ttranche/push_swap**
  2. https://github.com/MarieCHal/push_swap/tree/master/push_swap
  3. **https://github.com/VBrazhnik/Push_swap/wiki/Algorithm**
  4. **https://medium.com/@jamierobertdawson/push-swap-the-least-amount-of-moves-with-two-stacks-d1e76a71789a**
  5. https://www.linkedin.com/posts/fermatslibrary_a-visualization-of-different-sorting-algorithms-activity-6964204674228924416-8VxH/?utm_source=linkedin_share&utm_medium=ios_app
  6. https://www.youtube.com/watch?v=u-MsFaqntS0&t=800s
  7. https://vscza.itch.io/push-swap (unity ile yapılmış push swap oyunu)
  8. https://openbase.com/python/push-swap-gui (idk)
  9. https://github.com/o-reo/push_swap_visualizer
  10. https://medium.com/@msouiyeh/not-your-typical-42network-push-swap-cc583f863a90
  11. https://github.com/Victor-Akio/Push-Swap-42
  12. https://github.com/LuigiEnzoFerrari/linked-list
      bağlı listelerle ilgili örneklerin oldugu güzel bi repo
  13. https://github.com/LeoFu9487/push_swap_tutorial
  14. https://github.com/anyaschukin/Push_Swap
  15. https://github.com/wwatkins42/42_C_Projects.git
  16. https://github.com/byaliego/push_swap
  17. https://github.com/librity/ft_push_swap : muhteşem bi kaynak
  18. https://github.com/AdrianWR/push_swap
  19. https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html
  20. https://www.toptal.com/developers/sorting-algorithms
  21. https://medium.com/@ayogun/push-swap-c1f5d2d41e97
  22. https://medium.com/nerd-for-tech/push-swap-tutorial-fa746e6aba1e
  23. https://sharkigamers.github.io/pushswap_epitech_project/
      doubly circular linked list kullanarak yapmış. güzel anlatıyor ama çok karışık
  24. `42_born_2_code` [push_swap] topic

25. https://ytrack.learn.ynov.com/git/root/public/commit/1b7a4358438f25711e32b399b2b6e4741e047199
26. https://www.youtube.com/watch?v=M1hABszC9gU&list=PLh9ECzBB8tJPTWIUbZjHZMMGuZcpHUv5h&index=6

- push_swap projesinin amacı farklı sayılardaki sayıları algoritmalar kullanarak küçükten büyüğe doğru sıralamak. given a sequence of random numbers disposed in a stack data structure `stack A` sort them with the help of an empty stack `stack B` and a set of specific stack operations.
- push swap mantık olarak meşhur mantık sorusu hanoi kulesine benzer. bir çubuktaki farklı büyüklükteki diskleri bir diğer çubuğa büyükten küçüğe nasıl sıralayabilirsin sorusunu çözmeye çalışıyosun
- at the end, the smallest element should be on the top of the stack a and Stack b is empty.
- evo soruları
  - ./push_swap 3 2 1 çalıştır
  - ./push_swap 03 2 1 çalıştır
  - ./push_swap 3 02 1 çalıştır
  - https://www.calculatorsoup.com/calculators/statistics/random-number-generator.php
- kodun sayıları maksimum kaç hamlede çözmeli
  - Sorting 3 values: no more than 3 actions.

    Sorting 5 values: no more than 12 actions.

    Sorting 100 values: rating from 1 to 5 points depending on the number of actions:

    - 5 points for less than 700 actions
    - 4 points for less than 900
    - 3 points for less than 1100
    - 2 points for less than 1300
    - 1 point for less than 1500

    Sorting 500 values: rating from 1 to 5 points depending on the number of actions:

    - 5 points for less than 5500 actions
    - 4 points for less than 7000
    - 3 points for less than 8500
    - 2 points for less than 10000
    - 1 point for less than 11500

`TO DO LIST IN PUSH_SWAP`

- [ ] makefile yaratmalısın ve şu kurallar olmalı
  - [ ] $(NAME)

- [ ] CC
- [ ] CLAGS = -Wall -Werror -Wextra flagleriyle derlenmeli
- [ ] all
- [ ] clean
- [ ] fclean
- [ ] re
- [ ] .phony
- [ ] relink olmamalı

- [ ] push_swap.h yaz içerisinde stack tanımla ve kulllandığın fonksiyonları yaz
- [ ] operator fonksiyonlarını yaz
  - [ ] sa
  - [ ] sb
  - [ ] ss
  - [ ] pa
  - [ ] pb
  - [ ] ra
  - [ ] rb
  - [ ] rr
  - [ ] rra
  - [ ] rrb
  - [ ] rrr
  - [ ] sa, rb vs.'ler \n ile ayrılmalı
  - [ ] operatorleri kontrol et doğru çalıştıklarından emin ol.
- [ ] atoi() yaz. it'll be useful when we take the arguments from command line as char string and then turn them into integers.
- [ ] stack'ini initialize eden bir fonksiyon yazmalısın. bu fonksiyon stack'in her elemanını 0'a eşitlemeli.
- [ ] free()'leyip çıkan bir `free_all()` fonksiyonu yaz. bu fonksiyon yer ayırdığın tüm char stringleri freelemeli. ayrıca exit(1) demeden önce hata modunda write ile Error yazdirmalisin: `write(2, "error\n", 7);`
- [ ] main.c'de
  - [ ] ac kontrolü yap (./push_swap sayi1 sayi2) değilse
  - [ ] tüm argümanlari bir char str'ye atmalısın aralarına strjoin()'i optimize ederek boşluk koydurmalısın
  - [ ] av kontrolü yap

- [ ] öncelikle `void num_check(t_stack *data)` yapiyoruz ve içerinde yarattığımız bir stringe data→str'yi atiyoruz. daha sonra bu str'ye aşağıdaki kontrolleri gerçekleştiriyoruz.
    - [ ] argümanlar 0-9 aralığında, boşluk ya da +/- harici bir şey ise hata ver
    - [ ] iki artı ya da eksi üst üste gelirse hata ver (++, +-, -+, — …)
    - [ ] + ya da - sayıdan sonra gelirse hata ver (32+, 100- vs.)
    - [ ] + ya da -'den sonra \0, \t, ' ' gelirse hata ver
      bu 4ü string formda bütün hatalı durumları belirtir.
- [ ] sayıları stack_a'ya doldur.
    - [ ] INT_MIN'den küçük, INT_MAX'tan büyük sayi verilirse hata ver
        - [ ] (./push_swap `2147482648` 30) —> calısmamalı
        - [ ] (./push_swap `-2147483649` 30) —> calısmamalı
    - [ ] tekrar eden sayı verilirse hata ver
    - [ ] sayılar halihazırda sıralanmış gelirse hata ver
- [ ] sayıların sayısını bulan bir fonksiyon yaz (./push_swap 10 2 5 -20 300 —> 5)
- [ ] eğer 3, 4 ya da 5 argüman gelirse bunları radix olmadan sıralamalısın çünkü radix gereksiz uzatyor az sayıda sayılarda.

- [ ] radix() sıralama algoritmasını yazmalısın
    - [ ] radix en büyük sayının bit sayısı kadar adımda sıraladığı için en büyük sayının bit sayısını hesaplayan bir `max_bit()` fonksiyonu yazmalısın
    - [ ] daha sonra `radix()` yazpmalısın burada radixin formülü olmalı
- [ ] segmentation, bus error, double free vs. olmamalı
- [ ] memory leak olmamalı `valgrind` ile kontrol et
- [ ] testlerdan geç
    - [ ] https://github.com/lmalki-h/push_swap_tester
    - [ ] https://github.com/laisarena/push_swap_tester
    - [ ] https://github.com/minckim42/push_swap_tester
    - [ ] https://github.com/wwwwelton/push_swap/blob/master/tester.sh
    - [ ] https://github.com/VBrazhnik/Push_swap/blob/master/benchmark.sh
    - [ ] intrada'daki checkeri kullanmak için

Checker i indir sonra push swapin icine at terminalden `chmod uga+x checker_linux` yaz. Kullanmaya hazir kullanmak icin mesela `./pushswap 3 2 1 |  ./checker_linux 3 2 1` yazabilirsin. Diger alternatif `ARG="sayilar"; ./push_swap $ARG | ./checker_linux $ARG` **.** [sa, pb, rrr] vs. gibi verdiği argümanları kullanmak için de alt alta
sa
pb

rrr
yazıp sonra ctrl+d yapman gerekiyor.

- kullanman gereken komutlar
  - sa swap a - swap the first 2 elements at the top of stack a.
    Do nothing if there is only one or no elements).
    sb swap b - swap the first 2 elements at the top of stack b.
    Do nothing if there is only one or no elements).
    ss sa and sb at the same time.
    pa push a - take the first element at the top of b
    and put it at the top of a. Do nothing if b is empty.
    pb push b - take the first element at the top of a and
    put it at the top of b. Do nothing if a is empty.
    ra rotate a - shift up all elements of stack a by 1.
    The first element becomes the last one.
    mesela {20, 30, 42, 5} --> {30, 42, 5, 20} olur
    rb rotate b - shift up all elements of stack b by 1.
    The first element becomes the last one.
    rr ra and rb at the same time.
    rra reverse rotate a - shift down all elements of stack a by 1.
    The last element becomes the first one.
    mesela {20, 30, 42, 5} --> {5, 20, 30, 42} olur
    rrb reverse rotate b - shift down all elements of stack b by 1.
    The last element becomes the first one.
    rrr rra and rrb at the same time.
- push swap'ta sana ./push_swap `"3 4 5" 0 1 "2 7 8"` şeklinde (aynısının "sizi de geçerli) argüman geliyor. bu argüman

```
./push_swap 3 4 5 0 1 2 7 8
=>
    3
    4
    5
    0
    1
    2
    7
    8
    _    _
    a    b
```

usage example

```
$>./checker 3 2 1 0
rra
pb
sa
rra
```

```
pa
OK
$>./checker 3 2 1 0
sa
rra
pb
KO
$>./checker 3 2 one 0
Error
```

- The project can be done by multiple different way:
  - By using array.
  - By using simply linked list.
  - By using doubly linked list.
  - By using doubly circular linked list.
- Dont limit yourself to C use python or any other laguage that handles the memory stuff to iterate fast on the algorithm logic
- Once you tested your algorithm and it meets the requirements, implement the 'backend' memory managment stuff, SEPARATELY from the 'driver' algorithm that will orchestrate your backend
- Also on the general subject of optimization, finding the shortest possible amount of instructions, in this case. Take a look at dynamic programming, especially Longest Increasing Subsequence, it will help you a lot
- push_swap
  - You have to write a program named push_swap which will receive as an argument the stack a formatted as a list of integers. The first argument should be at the top of the stack (be careful about the order).
  - The program must display the smallest list of instructions possible to sort the stack a, the smallest number being at the top.
  - Instructions must be separaed by a ’\n’ and nothing else.
  - The goal is to sort the stack with the minimum possible number of operations. During defence we’ll compare the number of instructions your program found with a maximum number of operation tolerated. If your program either displays a list too big or if the list isn’t sorted properly, you’ll get no points.
  - In case of error, you must display Error followed by a ’\n’ on the standard error. Errors include for example: some arguments aren’t integers, some arguments are bigger than an integer, and/or there are duplicates.
  - c’de

    ```
    if (a > b > c);
    ```

diyemiyosun. çünkü a>b ifadesi 0 ya da 1 döndürüyor daha sonra 1 > c ifadesini ariyor. bunun için

```
if ((a > b) && (b > c));
```

demelisin.

-

SORTING ALGORITHMS

- bubble sort:
  - ilk olarak 1. ve 2. elemanları karşılaştırır. eğer 1 2'den büyükse swap eder. sonra 2-3 sonra 3-4 … diye size-1 kadar gider

```
void bubbleSort(int arr[], int size)
{
    int i = 0;
    while (i < size - 1)
    {
        if (arr[i] > arr[i + 1])
            swap(&arr[i], &arr[i + 1]); // &'sız hata veriyor
        i++;
    }
}
```

worst case scenario for bubble sort is when array is given already sorted but in a descending order

- insertion sort: useful for small data values.

mesela `12 11 13 5 6` diye bir dizimiz var. insertion sort önce 11 ile 12'yi karşılaştırır. eğer `arr[i] > arr[i+1]` ise onları `swap()` fonksiyonu yardımıyla yer değiştirir. sonra `arr[i+1] > tab[i+2]` karşılaştırması yapar (2. ve 3. && 3. ve 4. && 4. ve 5. …) böyle böyle devam eder sıralamaya. sona gelince geriye dönük karşılaştırma yapar. eğer sağdaki soldakinden küçükse onları da swap eder ve böylece array sıralanmış olur.

> 得 **Insertion Sort Algorithm**
>
> To sort an array of size N in ascending order:
>
> - Iterate from arr[1] to arr[N] over the array.
> - Compare the current element (key) to its predecessor.
> - If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

bu da koda dökülmüş hali

```
void insertionSort(int array[], int size) {
  for (int step = 1; step < size; step++) {
    int key = array[step];
    int j = step - 1;

    // Compare key with each element on the left of it until an element smaller than
    // it is found.
    // For descending order, change key<array[j] to key>array[j].
    while (key < array[j] && j >= 0) {
      array[j + 1] = array[j];
      --j;
    }
    array[j + 1] = key;
  }
}
```

- quick sort:
- merge sort: Think of it as a recursive algorithm continuously splits the array in half until it cannot be further divided. This means that if the array becomes empty or has only one element left, the dividing will stop, i.e. it is the base case to stop the recursion. If the array has multiple elements, split the array into halves and recursively invoke the merge sort on each of the halves. Finally, when both halves are sorted, the merge operation is applied. Merge operation is the process of taking two smaller sorted arrays and combining them to eventually make a larger one.
    - eleman sayısı 1’den büyük mü diye bakar
    - büyükse arrayi ortadan 2ye böler. 1 eleman kalana kadar 2ye bölmeye devam eder.
    - ilk 2’yi karşılaştırır küçüğü sağa atar büyüğü altına alır böyle böyle 2li gruplar oluşturur.
    - bu sefer bu ikili grupları 2ye ayırır ve ilk sayılarını karşılaştırır. alttaki ikilinin ilk sayısı üstteki ikilinin sayısından küçükse küçük olanı sağa alır.
    - bu sefer diğerlerini karşılaştırır ve min olanı sağdakinin altına alır.
    - kalan sayıyı da en alta alır ve sıralamış olur.
    - aynı işlemi arrayin 2. yarısı için de yapar.

```
// Merges two subarrays of arr[].
// First subarray is arr[l..m]
// Second subarray is arr[m+1..r]
void merge(int arr[], int l, int m, int r)
{
  int i, j, k;
  int n1 = m - l + 1;
  int n2 = r - m;

  /* create temp arrays */
  int L[n1], R[n2];

  /* Copy data to temp arrays L[] and R[] */
  for (i = 0; i < n1; i++)
    L[i] = arr[l + i];
  for (j = 0; j < n2; j++)
    R[j] = arr[m + 1 + j];
```

```
  /* Merge the temp arrays back into arr[l..r]*/
  i = 0; // Initial index of first subarray
  j = 0; // Initial index of second subarray
  k = l; // Initial index of merged subarray
  while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
      arr[k] = L[i];
      i++;
    }
    else {
      arr[k] = R[j];
      j++;
    }
    k++;
  }

  /* Copy the remaining elements of L[], if there
  are any */
  while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
  }

  /* Copy the remaining elements of R[], if there
  are any */
  while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
  }
}

/* l is for left index and r is right index of the
sub-array of arr to be sorted */
void mergeSort(int arr[], int l, int r)
{
  if (l < r) {
    // Same as (l+r)/2, but avoids overflow for
    // large l and h
    int m = l + (r - l) / 2;

    // Sort first and second halves
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);

    merge(arr, l, m, r);
  }
}

/* Function to print an array */
void printArray(int A[], int size)
{
  int i;
  for (i = 0; i < size; i++)
    printf("%d ", A[i]);
  printf("\n");
}

/* Driver code */
int main()
{
  int arr[] = { 12, 11, 13, 5, 6, 7 };
  int arr_size = sizeof(arr) / sizeof(arr[0]);

  printf("Given array is \n");
```

```
  printArray(arr, arr_size);

  mergeSort(arr, 0, arr_size - 1);

  printf("\nSorted array is \n");
  printArray(arr, arr_size);
  return 0;
}
```

- selection sort : The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning.
  - 0. indisteki sayıdan başlar ve smallest'a atar.
  - indisi arttirip daha küçük bir sayıya denk gelirse smallest'a bu sefer onu atar
  - bunu array bitene kadar yapar.
  - smallest'ı 0. indise atar.
  - bu sefer 0'dan değil 1. indisten başlar.
  - sayıların sıralamalarını bildiği için sürekli bir sonraki indise giderek bir öncekinden büyük en küçük sayıyla o anki indisi swap'lar (aynıysa hiçbir şey yapmaz).
  - merge sort is much more efficient than selection sort.

```
// selection sort in c

void swap(int *x, int *y);

void selectionSort(int arr[], int n)
{
  int i, j, min_idx;

  // One by one move boundary of unsorted subarray
  for (i = 0; i < n-1; i++)
  {
    // Find the minimum element in unsorted array
    min_idx = i;
    for (j = i+1; j < n; j++)
    if (arr[j] < arr[min_idx])
      min_idx = j;

    // Swap the found minimum element with the first element
    if(min_idx != i)
      swap(&arr[min_idx], &arr[i]);
  }
}
```

- radix sort: bitlere göre sıralar

170	45	75	90	802	24	2	66

  - yukarıda önce birler basamaklarına göre sıralar. basamağı aynı olan sayıları orijinal listedeki (her liste değiştikten sonra orijinal liste bir önceki liste olur, ilk liste değil) - konumlarına göre sıralar

(mesela 125 ve 15'i birler basamağına göre sıralarken 125 15'ten önce geldiyse ilk onu koyar)

    - daha sonra onlar, yüzler, binler basamaklarına göre sıralar. bunu en büyük sayının basamak sayısı kadar devam ettirir.

- radix kök sayı, taban demektir.
- algoritma 900 yıllık bi kavram. iranlı el-harezmi'den gelmekte. cebiri bulan adam. algebra da buradan geliyor. latinceye çevrilince al-khrawizmi algoritmi oluyor. o da algorithm'e dönüşmüş. algoritmi kelimesi ilk olarak ortaçağda decimal number sistemi için kullanılıyor hatta chaucer'de de geçiyor (franklin's tales). alan turing bir bilgsayara algoritma kurmayı öğretip kompleks matematik problemleri çözmesini sağladı.
- pseudo : benzeyen, ima eden demek
- radix formula in cpp

```
// C++ implementation of Radix Sort

#include <iostream>
using namespace std;

// A utility function to get maximum value in arr[]
int getMax(int arr[], int n)
{
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// A function to do counting sort of arr[] according to
// the digit represented by exp.
void countSort(int arr[], int n, int exp)
{
    int output[n]; // output array
    int i, count[10] = {0};

    // Store count of occurrences in count[]
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Change count[i] so that count[i] now contains actual
    //  position of this digit in output[]
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array
    for (i = n - 1; i >= 0; i--)
    {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the output array to arr[], so that arr[] now
    // contains sorted numbers according to current digit
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}

// The main function to that sorts arr[] of size n using
```

```
// Radix Sort
void radixsort(int arr[], int n)
{
    // Find the maximum number to know number of digits
    int m = getMax(arr, n);

    // Do counting sort for every digit. Note that instead
    // of passing digit number, exp is passed. exp is 10^i
    // where i is current digit number
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

// A utility function to print an array
void print(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// Driver Code
int main()
{
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function Call
    radixsort(arr, n);
    print(arr, n);
    return 0;
  }
```

- Given that we have 0 or 1 numbers at the stack, we don't need to do anything, as we can assume that a stack of a single number is already sorted. At 2 numbers, we may have only two scenarios: The stack is already sorted; or if not, swap the numbers of stack A. When we have 3 numbers, the situation is a little more complex, but it's still easy. In this case, a sequence of 3 numbers only allow 3! = 6 permutations of elements. Given that one these permutations is the sorted sequence, we only have to bother with 5 permutations, each one of them with a different set of instructions required to sort. When we have 4 or 5 elements, we just need to push the top two elements to stack B and run the sorting algorithm on the remaining stack. To finish the sorting process, we just need to push the last two or less elements on stack B, finding the correct position on the stack A before running the push operation.
- radix pozitif sayılarda işe yaramıyor o yüzden number simplifying kullanmamız lazım. mesela

```
87 -487 781 -100 101 0 1
```

gibi bir sayı geldiğinde bunları 0-n aralığında sayılara dönüştürmemiz gerekiyor

```
4 0 6 1 5 2 3
```

number simplifier formülü

```
vector<int> input;
//parse the numbers into input ...
```

```
//reminder : there is no duplicate
vector<int> copy = input; // copy the numbers from input
sort(copy.begin(), copy.end());
for(int i = 0 ; i < input.size() ; ++i)
    for(int j = 0 ; j < copy.size() ; ++j)
        if (input[i] == copy[j])
            input[i] = j;
//put input into stack a ...

/*
Remark : This is actually not the most efficient
way to implement them. If you want a more efficient
program, please learn about binary search
or unordered_map (hash)
*/
```

kullanımı

```
void sort(int *arr, unsigned int size)
{
    int i = 0;
    int i2 = 0;
    int tmp;
    while (i < size - 1)
    {
        while (i2 < size - 1)
        {
            if (arr[i2] > arr[i2 + 1])
            {
                tmp = arr[i2];
                arr[i2] = arr[i2 + 1];
                arr[i2 + 1] = tmp;
            }
            i2++;
        }
        i2 = 0;
        i++;
    }
}

int main()
{
    int arr[5] = {10, 3, 6, 50, 40};
    int arr2[5];
    for (int i = 0; i < 5; i++)
        arr2[i] = arr[i];
    sort(arr2, 5);
    for (int i = 0; i < 5; ++i)
        for (int j = 0; j < 5; ++j)
            if (arr[i] == arr2[j])
                arr[i] = j;
    printf("basitleştirilmiş sayı sıralaması:\n");
    for (int i = 0; i < 5; i++)
        printf("[%d]", arr[i]);
    printf("\nbasitleştirilmemiş sayı sıralaması:\n");
    for (int i = 0; i < 5; i++)
        printf("[%d]", arr2[i]);
} >>>
basitleştirilmiş sayı sıralaması:
[2][0][1][4][3]
basitleştirilmemiş sayı sıralaması:
[3][6][10][40][50]
```

- digits
  - most significant digit : sayının en solundaki digit
  - least significant digit : sayının en sağındaki digit (her zaman birler basamağı)
- radix algoritması
  - radix'e gelene kadar `already_sorted()` ve `num_checker()` kontrolleri yapilmiş oluyor.
  - sayıları önce least digitlerine göre ayır (birler basamağı 0-9 olanlar bir yerde)
  - daha sonra sayıları bunlara göre sırala (birler basamağı aynı olan sayıları orijinal listedeki occurence sıralarına göre sırala)
  - daha sonra birler basamağına göre sıralanmış bu liste aynı şeyi onlar basamağına göre yap (basamağı olmayanların basamağı 0 kabul edilir mesela 25'in yüzler basamağı 0 alınır)
  - daha sonra eğer 3 haneli bir sayı var ise listede aynı şeyi yüzler basamağı için de yap. thats it. the list is now all sorted.
- for every bit, & operatör works as

```
1 & 1 == 1
1 & 0 == 0
0 & 1 == 0
0 & 0 == 0
```

mesela printf("%d", 6&5) demiş olalım.

```
// 6 = 0000 0110
// 5 = 0000 0101
//     ----------
// x = 0000 0100
// x = 4
```

⛔ çift sayıların 1 ile &'i 0, tek sayıların 1 verir

- radix nası çalışır?
  - iki tane stack'in olsun. a ve b.

```
100
000
110
001
101
010
011
----------     ----------
a              b
```

bunları öncelikle least-significant-digit'lerine yani en sağdaki basamaklarına göre b stackinde gruplamamız gerek. bunun için eğer üstteki sayının (100) sağındaki bit 0 ise (evet) o zaman pb ile b'ye atiyoruz. değilse ra diyerek (her sayıyı bir üste kaydırarak) bu sayıyı atlıyoruz. bunu a'daki eleman sayısı kadar yapınca sağdaki biti 0 olanları b stackinde toplamış oluyoruz

```
               010
001            110
101            000
011            100
----------     ----------
a              b
```

- daha sonra pa (put to a) ile b'deki sayıları a'ya geri gönderiyoruz. son hali böyle

```
100
000
110
010
001
101
011
----------     ----------
a              b
```

- bu işlemleri 2. bit, 3. bit için de yapıyoruz. totalde en büyük sayının bit sayısı kadar yapiyoruz bu işlemi (5 bitse 5 kere). en sonunda pa ile her şeyi a'ya attiğimizda sıralanmış oluyor.
- bit sayısı hesaplayan fonksiyon

```
unsigned int countBits(unsigned int n)
{
    unsigned int count = 0;
    while (n)
    {
        count++;
        n >>= 1;
    }
    return count;
}
```

- radix'in C++ kodu:

```
int size = a.size();
int max_num = size - 1; // biggest number in 'a' is stack_size-1
int max_bits = 0;       // number of bits of the max value
while ((max_num >> max_bits) != 0)
    max_bits++;
for (int i = 0; i < max_bits; ++i) // repeat for max_bits times
{
    for (int j = 0; j < size; ++j)
    {
        int num = a.top(); // top number of A

        // if the (i + 1)-th bit is 1, leave in stack a
        /* The result of (num>>i)&1 must be 0 or 1, which is the value of the (i + 1)-th bit of num.
        For example, (5>>2)&1 == 1 because
        5       : 101
        5>>2    :   1
        (5>>2)&1 :  1

        (8>>2)&1 == 0 because
        8       : 1000
        8>>2    :   10
        (8>>2)&1 :   0  */
        if ((num >> i) & 1 == 1)
            ra();
        // otherwise push to stack b
        else
            pb();
    }
    // put into boxes done
    while (!b.empty())
        pa(); // while stack b is not empty, do pa
    // connect numbers done
}
```

- Instructions must be separaed by a '\n' and nothing else. In case of error, you must display Error followed by a '\n' on the standard error. Errors include for example: some arguments aren't integers, some arguments are bigger than an integer, and/or there are duplicates.
-
-

STACKS

- A stack is a linear data structure, collection of items of the same type. Stack follows the Last In First Out (LIFO) fashion wherein the last element entered is the first one to be popped out. Stacks can be of two types: static and dynamic stack. A static stack is implemented using an array in C, whereas a dynamic stack is implemented using a linked list in C.

```
// basic implementation of linked lists in c
struct node
{
  int data;
  struct node *next;
};
```

The following are the basic operations served by the Stacks.

  - Push: This function adds an element to the top of the Stack.
  - Pop: This function removes the topmost element from the stack.
  - IsEmpty: Checks whether the stack is empty.
  - IsFull: Checks whether the stack is full.
  - Top: Displays the topmost element of the stack.
- Stacks can be represented using structures, pointers, arrays, or linked lists.

💡 The operations work as follows:

1. A pointer called is used to keep track of the top element in the stack.

   TOP

2. When initializing the stack, we set its value to -1 so that we can check if the stack is empty by comparing `TOP == -1`.
3. On pushing an element, we increase the value of and place the new element in the position pointed to by .

   TOP

   TOP

4. On popping an element, we return the element pointed to by and reduce its value.

   TOP

5. Before pushing, we check if the stack is already full
6. Before popping, we check if the stack is already empty

listelerle kullanımı:

```
// C program for linked list implementation of stack
#include <limits.h>
```

```c
#include <stdio.h>
#include <stdlib.h>

// A structure to represent a stack
struct StackNode {
  int data;
  struct StackNode* next;
};

struct StackNode* newNode(int data)
{
  struct StackNode* stackNode =
  (struct StackNode*)
  malloc(sizeof(struct StackNode));
  stackNode->data = data;
  stackNode->next = NULL;
  return stackNode;
}

int isEmpty(struct StackNode* root)
{
  return !root;
}

void push(struct StackNode** root, int data)
{
  struct StackNode* stackNode = newNode(data);
  stackNode->next = *root;
  *root = stackNode;
  printf("%d pushed to stack\n", data);
}

int pop(struct StackNode** root)
{
  if (isEmpty(*root))
    return INT_MIN;
  struct StackNode* temp = *root;
  *root = (*root)->next;
  int popped = temp->data;
  free(temp);

  return popped;
}

int peek(struct StackNode* root)
{
  if (isEmpty(root))
    return INT_MIN;
  return root->data;
}

int main()
{
  struct StackNode* root = NULL;

  push(&root, 10);
  push(&root, 20);
  push(&root, 30);

  printf("%d popped from stack\n", pop(&root));

  printf("Top element is %d\n", peek(root));

  return 0;
}
```

structla kullanımı:

```
// Stack implementation in C

#include <stdio.h>
#include <stdlib.h>

#define MAX 10

int count = 0;

// Creating a stack
struct stack {
  int items[MAX];
  int top;
};
typedef struct stack st;

void createEmptyStack(st *s) {
  s->top = -1;
}

// Check if the stack is full
int isfull(st *s) {
  if (s->top == MAX - 1)
    return 1;
  else
    return 0;
}

// Check if the stack is empty
int isempty(st *s) {
  if (s->top == -1)
    return 1;
  else
    return 0;
}

// Add elements into stack
void push(st *s, int newitem) {
  if (isfull(s)) {
    printf("STACK FULL");
  } else {
    s->top++;
    s->items[s->top] = newitem;
  }
  count++;
}

// Remove element from stack
void pop(st *s) {
  if (isempty(s)) {
    printf("\n STACK EMPTY \n");
  } else {
    printf("Item popped= %d", s->items[s->top]);
    s->top--;
  }
  count--;
  printf("\n");
}

// Print elements of stack
void printStack(st *s) {
  printf("Stack: ");
  for (int i = 0; i < count; i++) {
```

```
    printf("%d ", s->items[i]);
  }
  printf("\n");
}

// Driver code
int main() {
  int ch;
  st *s = (st *)malloc(sizeof(st));

  createEmptyStack(s);

  push(s, 1);
  push(s, 2);
  push(s, 3);
  push(s, 4);

  printStack(s);

  pop(s);

  printf("\nAfter popping out\n");
  printStack(s);
}
```

arrayle kullanımı:

```
// C program for array implementation of stack
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// A structure to represent a stack
struct Stack {
  int top;
  unsigned capacity;
  int* array;
};

// function to create a stack of given capacity. It initializes size of
// stack as 0
struct Stack* createStack(unsigned capacity)
{
  struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
  stack->capacity = capacity;
  stack->top = -1;
  stack->array = (int*)malloc(stack->capacity * sizeof(int));
  return stack;
}

// Stack is full when top is equal to the last index
int isFull(struct Stack* stack)
{
  return stack->top == stack->capacity - 1;
}

// Stack is empty when top is equal to -1
int isEmpty(struct Stack* stack)
{
  return stack->top == -1;
}

// Function to add an item to stack. It increases top by 1
```

```
void push(struct Stack* stack, int item)
{
  if (isFull(stack))
    return;
  stack->array[++stack->top] = item;
  printf("%d pushed to stack\n", item);
}

// Function to remove an item from stack. It decreases top by 1
int pop(struct Stack* stack)
{
  if (isEmpty(stack))
    return INT_MIN;
  return stack->array[stack->top--];
}

// Function to return the top from stack without removing it
int peek(struct Stack* stack)
{
  if (isEmpty(stack))
    return INT_MIN;
  return stack->array[stack->top];
}

// Driver program to test above functions
int main()
{
  struct Stack* stack = createStack(100);

  push(stack, 10);
  push(stack, 20);
  push(stack, 30);

  printf("%d popped from stack\n", pop(stack));

  return 0;
}
```

- The back button in a browser saves all the URLs you have visited previously in a stack. Each time you visit a new page, it is added on top of the stack. When you press the back button, the current URL is removed from the stack, and the previous URL is accessed.
- örnek

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/0058e239-f743-4392-8565-70df029342aa/a.c

- radix bitlere göre sıralarken eğer o an baktığı bit 1se ra yapar 0sa pb yapar tüm sayılar işlem görünce stack_b bitene kadar pa yapar. sonra bir sonraki bite geçer ve böyle böyle maks bit kadar işlem yapar.